

转生指南——PY / C++ 语法对照

转生指南——PY / C++ 语法对照

写在前面

一、基本常识

二、输入输出

1.基础

2.特殊

三、流程控制

四、常用容器对照

1. 列表

特殊操作

2.字典/映射

3.集合

4. 优先队列(没啥用了，可以被multiset平替)

5.自定义结构

五、函数定义与递归

六、其他常用算法库对照

1.二分

2.最值

3.数学

七、字符串处理

八、DEBUG

写在前面

本转生指南为EH赛时自用，记录了一些常见PY与C++的语法知识。

一、基本常识

功能	Python	C++
注释	<code># 注释</code>	<code>// 注释</code> ，多行： <code>/* */</code>
变量定义	<code>a = 5</code>	<code>int a = 5;</code>

二、输入输出

1.基础

功能	Python	C++	举例
输入一个整数	<code>x = int(input())</code>	<code>int x; cin >> x;</code>	5
输入一行多个整数	<code>a, b = map(int, input().split())</code>	<code>int a, b; cin >> a >> b;</code>	1 2
快速IO(直接加即可)	<code>import sys input = sys.stdin.readline()</code>	<code>ios::sync_with_stdio(false); cin.tie(0);</code>	-
输入一列整数	<code>a = list(map(int, input().split()))</code>	<code>for (int i=0; i < n; i++) { cin >> a[i]; }</code>	1 2 3 4
输出变量	<code>print(a)</code>	<code>cout << a << endl;</code>	-
输出一列“整数”	<code>print(*a) or print(" ".join(map(str, a)))</code>	<code>for (int i=0; i < n; i++) {cout << a[i] << " "; }</code>	1 2 3 4
输出一列整数	<code>print("".join(map(str, a)))</code>	<code>for (int i=0; i < n; i++) {cout << a[i]; }</code>	1234
读字符串作数组	<code>s = list(input().strip())</code>	<code>char c6[5]; scanf("%s", c6);</code>	abcde

2.特殊

输出固定小数位数：

```
#include<bits/stdc++.h>
using namespace std;
int main(){
    vector<vector<int>> a(3);
    double pi = 11.4514114514;
    cout << setiosflags(ios::fixed) << setprecision(6) << pi;
}
```

```
s = 11.4514114514
print("{:.6f}".format(s))
```

三、流程控制

功能	Python	C++
条件语句	<pre>if x > 0: elif x == 0: else:</pre>	<pre>if (x > 0) {} else if (x == 0) {} else {}</pre>
for循环	<pre>for i in range(n): for i in range(n - 1, -1, -1): for x in edge[i]: for u, v in edges:</pre>	<pre>for (int i = 0; i < n; ++i) for (int i = n - 1; i > -1, i--) for (auto x : edge[i]) for (auto [u, v] : edges)</pre>
while循环	<pre>while x < 10:</pre>	<pre>while (x < 10)</pre>
循环中断	<pre>break, continue</pre>	<pre>break;, continue;</pre>
退出程序	<pre>exit(0)</pre>	<pre>return 0</pre>

四、常用容器对照

1. 列表

py与C++的各函数的时间复杂度一致。以下不加赘述。

基本操作

功能	Python (列表迭代式yyds)	C++
定义/初始化	<pre>matrix = [[0 for i in range(n + 1)] for j in range(n + 1)] a = [1, 2, 3] b = [0] * n</pre>	<pre>vector<vector<int>> matrix(n + 1, vector<int>(n + 1, 0)); vector<int> a = {1, 2, 3}; vector<int> b(n, 0); vector<vector<vector<int>>> matrix(n + 1, vector<vector<int>>(n + 1, vector<int>(n + 1, 0)));</pre>
添加元素	<pre>a.append(4) a.insert(1, 1.5)</pre>	<pre>a.push_back(4); a.insert(a.begin()+1, 1.5);</pre>
访问元素	<pre>val = a[0] val = a[-1]</pre>	<pre>int val = a[0]; int val = a.back();</pre>
修改元素	<pre>a[0] = 10</pre>	<pre>a[0] = 10;</pre>

删除操作

功能	Python	C++
删除末尾元素	<code>a.pop()</code>	<code>a.pop_back();</code>
删除指定位置	<code>a.pop(0)</code>	<code>a.erase(a.begin());</code>
删除指定值	<code>a.remove(2)</code>	<code>a.erase(remove(a.begin(), a.end(), 2), a.end());</code>
删除范围	<code>del a[0:2]</code>	<code>a.erase(a.begin(), a.begin()+2);</code>

查询与属性

功能	Python	C++
大小	<code>len(a)</code>	<code>a.size()</code>
是否为空	<code>if len(a) == 0:</code>	<code>if (a.empty())</code>
查找元素	<code>if 2 in a:</code> <code>idx = a.index(2)</code>	<code>find(a.begin(), a.end(), 2) != a.end()</code> <code>find(a.begin(), a.end(), 2) - a.begin()</code>
计数	<code>cnt = a.count(2)</code>	<code>cnt = count(a.begin(), a.end(), 2);</code>

特殊操作

功能	Python	C++
清空列表	<code>a.clear()</code>	<code>a.clear();</code>
切片操作	<code>sub = a[1:3]</code>	<code>vector<int> sub(a.begin()+1, a.begin()+3);</code>
列表连接	<code>c = a + b</code>	<code>vector<int> c(a);</code> <code>c.insert(c.end(), b.begin(), b.end());</code>
排序	<code>a.sort()</code>	<code>sort(a.begin(), a.end());</code>
反转	<code>a.reverse()</code>	<code>reverse(a.begin(), a.end());</code>

重要操作——自定义sort排序:

```
a = [
    [5, 2],
    [4, 3],
    [8, 6]
]
# a.sort(key=lambda x: x[1]) # 按照第二关键字排序
# a.sort(key=lambda x: (x[1], x[0])) # 先按照第二关键字，再按照第一关键字排序
def cmp(k):
    a = k[0]
    b = k[1]
    return a + b

a.sort(key=lambda x: (cmp(x), x[0])) # 先按照cmp结果排序，再按照第一关键字排序

print(a)
```

```

#include <iostream>
#include <vector>
#include <algorithm>
#include <tuple>
using namespace std;

int main() {
    vector<vector<int>> a = {{5,2}, {4,3}, {8,6}};

    auto cmp = [](auto x) { return x[0] + x[1]; };

    sort(a.begin(), a.end(), [&](auto x, auto y) {
        int sum_x = cmp(x);
        int sum_y = cmp(y);
        return tie(sum_x, x[0]) < tie(sum_y, y[0]);
    });

    for (auto v : a) {
        cout << v[0] << " " << v[1] << "\n";
    }
}

```

🤖, 还有吗? 没了吧。

2.字典/映射

基本操作对比

功能	Python (dict)	C++ (std::map)
定义	<code>d = {}</code> <code>d = {'a':1, 'b':2}</code>	<code>map<string,int> d;</code> <code>map<string,int> d = {"a",1},{"b",2};</code>
插入/更新	<code>d['c'] = 3</code>	<code>d["c"] = 3;</code>
访问	<code>val = d['a']</code>	<code>int val = d["a"];</code>
检查存在	<code>if 'a' in d:</code>	<code>if (d.find("a") != d.end())</code>
删除	<code>del d['a']</code>	<code>d.erase("a");</code>
大小	<code>len(d)</code>	<code>d.size()</code>
清空	<code>d.clear()</code>	<code>d.clear();</code>
遍历	<code>for k in d:</code> <code>for k,v in d.items():</code>	<code>for (auto [k,v] : d)</code>

性能特性对比

特性	Python dict	C++ map (红黑树)
底层实现	哈希表	红黑树
插入/查找时间	O(1) 平均	O(log n)
内存使用	较高	中等
元素顺序	保持插入顺序 (Python 3.7+)	按键排序
适用场景	通用键值存储	需要有序遍历

Python 特殊——defaultdict，自动初始化：

```
from collections import defaultdict
d = defaultdict(list)
d[1].append(2)
d2 = defaultdict(int)
d2[1] += 1
print(d, d2)
```

C++特殊——map：

```
map<int, string, greater<int>> myMap; // 降序排序
auto it = next(myMap.begin(), 1); // 访问第二个元素
int main() {
    map<int, int> a;
    a[1] = 0;
    a[3] = 5;
    a[2] = 3;
    auto it = next(a.begin());
    cout << it ->first << it->second; // 找第一个元素
}
```

自定义排序的话，用列表sort吧EH，你毕竟不熟悉C++，还是别学太多了。

3.集合

基本操作对比

功能	Python (set)	C++ (std::set)
定义	<pre>s = {1, 2, 3} s = set()</pre>	<pre>set<int> s = {1, 2, 3}; set<int> s;</pre>
添加元素	<pre>s.add(4)</pre>	<pre>s.insert(4);</pre>
删除元素	<pre>s.remove(3) s.discard(3) (这个删除不存在的元素不会报错)</pre>	<pre>s.erase(3);</pre>
检查存在	<pre>if 3 in s:</pre>	<pre>if (s.find(3) != s.end())</pre>
大小	<pre>len(s)</pre>	<pre>s.size()</pre>
清空	<pre>s.clear()</pre>	<pre>s.clear();</pre>
遍历	<pre>for x in s:</pre>	<pre>for (auto x : s)</pre>

集合运算对比

功能	Python	C++ (set)
并集	<pre>s3 = s1 s2 s1.union(s2)</pre>	<pre>set<int> result; set_union(s1.begin(), s1.end(), s2.begin(), s2.end(), inserter(result, result.begin()));</pre>
交集	<pre>s1 & s2 s1.intersection(s2)</pre>	<pre>set_intersection(...) (同并集用法)</pre>
差集	<pre>s1 - s2 s1.difference(s2)</pre>	<pre>set_difference(...) (同并集用法)</pre>

性能特性对比

特性	Python set	C++ set (红黑树)
底层实现	哈希表	红黑树
插入/查找时间	O(1) 平均	O(log n)
内存使用	较高	中等
元素顺序	无序 (Python 3.7+)	按键排序
适用场景	通用集合操作	需要有序遍历

特殊——C++ `std::multiset` 详解

常用操作

操作	代码示例	说明
初始化	<code>multiset<int> ms = {3,1,4,1};</code>	创建包含重复元素的multiset
插入元素	<code>ms.insert(2);</code> <code>ms.insert({5,5,5});</code>	插入单个或多个元素
删除元素	<code>ms.erase(1);</code> <code>ms.erase(ms.find(3));</code> (find为 $O(\log n)$)	删除所有值为1的元素 删除第一个值为3的元素
计数元素	<code>int cnt = ms.count(5);</code>	返回值为5的元素个数k, 复杂度 $O(k + \log n)$
边界查找 (真神)	<code>multiset<int> ms = {1, 2, 3, 3, 3, 4, 5};</code> <code>auto it1 = ms.lower_bound(3);</code> <code>auto it2 = ms.upper_bound(3);</code> <code>cout << "lower_bound(3): " << *it1</code> <code><< "\n"; // 输出 3</code> <code>cout << "upper_bound(3): " << *it2</code> <code><< "\n"; // 输出 4</code>	第一个 ≥ 3 的元素 第一个 > 3 的元素
遍历	<code>for(auto elem : ms)</code>	有序遍历所有元素
大小/空检查	<code>bool e = ms.empty();</code> <code>int s = ms.size();</code>	检查是否为空 获取元素总数

```
s = set()
s.add((1, 6)) # set里可以塞元组，但是不能塞数组啥的
```

```
#include <iostream>
#include <set>
using namespace std;
int main(){
    multiset<int> ms = {1, 1, 4, 5, 1, 4};
    for (int i = 0; i < 2e5; i++)
    {
        ms.insert(i);
        int mi = *ms.begin();
        int ma = *ms.rbegin();
    }
}
```



```

multiset<int, greater<int>> ms = {3, 1, 4, 2, 5, 3}; // 实现逆向排序

int main() {
    set<std::pair<int, int>> coords;

    coords.emplace(3, 4);
    coords.emplace(1, 2);

    for (const auto& p : coords) {
        cout << "(" << p.first << ", " << p.second << ")\\n";
    }
}
// set里也可以塞pair

```

4. 优先队列(没啥用了, 可以被multiset平替)

在py里, 优先队列需要使用 `import heapq` 构建, 只有最小堆, 底层数据结构依赖为列表, 但是可以传入列表, 默认按位比较。

这里的東西有点杂, 不列表格了吧。

```

import heapq

heap = [] # 我嘞个初始化
heapq.heappush(heap, 3)
heapq.heappush(heap, 1)
heapq.heappush(heap, 4)

# q = []
# heapq.heappush(heap, [1, 4]) # 塞列表时, 默认比较第一位数

print(heapq.heappop(heap)) # 弹出顶部元素
print(heap[0]) # 查询顶部元素

# 假如需要最大堆的话, 只能全部使用负数, 切忌正负混合, 不然优先队列会出错。

```

C++默认最大堆。

```

int main() {
    // 最小堆 (需要greater比较器)
    priority_queue<int, vector<int>, greater<int>> min_heap;

    min_heap.push(3);
    min_heap.push(1);
    min_heap.push(4);

    std::cout << min_heap.top() << "\\n"; // 输出1 (最小元素)
    min_heap.pop(); // 移除最小元素
}

```

5.自定义结构

```
class MyClass:
    def __init__(self, a, b, c):
        self.a = a
        self.b = b
        self.c = c

    def __lt__(self, other): # 重载了比较运算符
        return (self.a + self.b + self.c) < (other.a + other.b + other.c)

tr = [MyClass(0, 0, 0) for i in range(4)]
tr[2].a = 5
tr.sort()
for i in range(4):
    print(tr[i].a, tr[i].b, tr[i].c)

# 结果:
# 0 0 0
# 0 0 0
# 0 0 0
# 5 0 0
```

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;
struct MyClass {
    int a, b, c;

    // 构造函数，用于初始化
    MyClass(int a = 0, int b = 0, int c = 0) : a(a), b(b), c(c) {}

    // 重载比较运算符，用于排序
    bool operator<(const MyClass& other) const {
        return (a + b + c) < (other.a + other.b + other.c);
    }
};

int main() {
    vector<MyClass> tr(4);
    tr[2].a = 5;

    sort(tr.begin(), tr.end());

    for (int i = 0; i < 4; ++i) {
        cout << tr[i].a << " " << tr[i].b << " " << tr[i].c << endl;
    }

    return 0;
}
```

自定义的结构，加上重载比较运算符后，就可以载入自动排序里去了。

五、函数定义与递归

功能	Python	C++
定义函数	<pre>def add(x, y): return x+y</pre>	<pre>int add(int x, int y) { return x + y; }</pre>
递归	较深递归需要 <code>sys.setrecursionlimit</code> 控制，Python3的递归比PyPy3要快，可以考虑使用Python3提交。	没有明显的限制

六、其他常用算法库对照

1.二分

```
import bisect

# 排序列表
lst = [1, 3, 4, 4, 6, 8]

# bisect_left - 返回第一个大于等于的位置，即返回插入位置，保持原有顺序
index = bisect.bisect_left(lst, 4) # 返回2
index = bisect.bisect_left(lst, 5) # 返回4

# bisect_right/bisect - 返回第一个大于的位置，即返回最右插入位置
index = bisect.bisect_right(lst, 4) # 返回4
index = bisect.bisect(lst, 4)      # 同上，返回4
```

```
#include <algorithm>
#include <vector>

std::vector<int> vec = {1, 3, 4, 4, 6, 8};

// lower_bound - 类似bisect_left, 返回第一个不小于值的迭代器
auto it = std::lower_bound(vec.begin(), vec.end(), 4); // 指向第一个4
it = std::lower_bound(vec.begin(), vec.end(), 5);      // 指向6

// upper_bound - 类似bisect_right, 返回第一个大于值的迭代器
it = std::upper_bound(vec.begin(), vec.end(), 4);      // 指向6

// binary_search - 检查值是否存在
bool exists = std::binary_search(vec.begin(), vec.end(), 4); // true
```

2.最值

```
a = [3, 1, 4, 1, 5, 9, 2, 6]

max_val = max(a) # 直接返回最大值 9
print("Max element:", max_val)

# 获取最大值的索引
max_index = a.index(max_val)
print("Position:", max_index) # 输出 5
```

```

#include <algorithm>
#include <vector>
#include <iostream>

int main() {
    std::vector<int> a = {3, 1, 4, 1, 5, 9, 2, 6};

    // 返回最大元素的迭代器
    auto it = std::max_element(a.begin(), a.end());

    if (it != a.end()) {
        std::cout << "Max element: " << *it << std::endl; // 输出 9
        std::cout << "Position: " << (it - a.begin()) << std::endl; // 输出 5
    }

    return 0;
}

```

3.数学

功能分类	Python	C++
基础数学	<code>math</code>	<code><cmath></code>
最大公约数	<code>math.gcd(a,b)</code>	<code>gcd(a,b)</code> (C++17 <code><numeric></code>)
最小公倍数	<code>math.lcm(a,b)</code> (Python 3.9+)	<code>lcm(a,b)</code> (C++17 <code><numeric></code>)
幂运算	<code>pow(x,y)</code> , <code>pow(x,y,mod)</code>	<code>pow(x,y)</code> , 快速幂需 手写
浮点精度	<code>double</code> , 非常飞舞, 但是可以注射血清进行强化- 比如扩展到200位: <code>from decimal import Decimal, getcontext</code> <code>getcontext().prec = 200</code>	<code>double / long</code> <code>double</code>
组合数	<code>math.comb(r,n)</code> 仅限于小一点的数, 大数太慢了	无
字符串转整数 突破上限	<code>import sys</code> <code>sys.set_int_max_str_digits(100001)</code>	无

角度相关:

```
#include <iostream>
#include <cmath>

int main() {
    double radians = M_PI / 4; // 45度对应的弧度
    std::cout << "sin( $\pi/4$ ): " << std::sin(radians) << std::endl; // 输出 0.707107

    double degrees = 45.0;
    double rad = degrees * M_PI / 180.0; // 角度转弧度
    std::cout << "cos(45°): " << std::cos(rad) << std::endl;
}
```

```
import math

radians = math.pi / 4 # 45度对应的弧度
print(math.sin(radians)) # 输出 0.7071067811865476

degrees = 45.0
rad = math.radians(degrees) # 角度转弧度（使用内置函数）
print(math.cos(rad)) # 输出 0.7071067811865476

# 反函数示例
print(math.degrees(math.asin(0.5))) # 输出 30.0（弧度转角度）
```

C++里的角度需要自己计算，所有东西都是弧度制。acos还有asin的精度需要注意，这俩的精度不是很好，能不用就不用。

七、字符串处理

常用操作

功能	Python	C++ (std::string)
定义	<code>s = "114514"</code> <code>s = ""</code>	<code>string s = "Hello";</code> <code>string s;</code>
输入，遇到空格停下	<code>string s; cin >> s;</code>	<code>s = input()</code>
字符串连接(不推荐)	<code>s += "a"</code>	<code>s += "a";</code>
求长度	<code>len(s)</code>	<code>s.length()</code>
检查存在，没有返回-1	<code>s.find("kl")</code>	<code>int a = s.find("ll");</code>
插入（第四个后面插入）	<code>s.insert(4, "a")</code>	<code>s.insert(4, "a")</code>
修改	不让修改	<code>s[1]='a';</code> 单引号
遍历	<code>for x in s:</code>	<code>for (auto x : s)</code>
截取	<code>s = s[1:3]</code>	<code>s = s.substr(1, 2);</code>
反转	<code>s = s[::-1]</code>	<code>reverse(s.begin(), s.end());</code>

```
// 输入的用法：
#include<bits/stdc++.h>
```

```

using namespace std;
int main(){
    vector<vector<int>> a(3);
    for (int i = 0; i < 3; i++)
    {
        string s;
        cin >> s;
        for (auto x : s){
            a[i].push_back(x - '0');
        }
    }
    for (int i = 0; i < 3; i++)
    {
        for (int j = 0; j < a[i].size(); j++){
            cout << a[i][j] << " ";
        }
        cout << "\n";
    }
}

```

八、DEBUG

对于py, 直接print就行了。

对于C++, 可以使用以下代码:

```

#ifdef LOCAL
template<class T> string ts(T v){stringstream ss;ss<<v;return ss.str();}
template<class A,class B> string ts(pair<A,B> p){return "
("+ts(p.first)+","+ts(p.second)+")";}
template<class T> string ts(vector<T> v){string s="{";for(auto
&x:v)s+=ts(x)+",";return s+"}";}
template<class T> string ts(set<T> v){string s="{";for(auto
&x:v)s+=ts(x)+",";return s+"}";}
template<class K,class V> string ts(map<K,V> m){string s="{";for(auto
&kv:m)s+=ts(kv)+",";return s+"}";}
template<class T> string ts(multiset<T> v){string s="{";for(auto
&x:v)s+=ts(x)+",";return s+"}";}
template<class T, size_t N> string ts(T (&a)[N]){string s="{";for(size_t
i=0;i<N;i++)s+=ts(a[i])+",";return s+"}";}
void dbg_out(){cerr<<"\n";}
template<class H,class...T> void dbg_out(H h,T...t){cerr<<" "
<<ts(h);dbg_out(t...);}
#define debug(...) cerr<<"["<<#__VA_ARGS__<<"]:"<<,dbg_out(__VA_ARGS__)
#else
#define debug(...) 114514
#endif

```